EstateXAi

AI-Driven Real Estate & PG/Hostel Platform

Project Report

Submitted by

Ayush Narkhede

Date: 19 October 2026

Table of Contents

Table of Contents 2

1. Abstract 5

2. Introduction 6

2.1 Project Overview 6

2.2 Problem Statement 6

2.3 Objectives 6

2.4 Technology Stack 7

2.5 Scope of the Project 8

3. System Analysis 10

3.1 Existing System Analysis 10

3.2 Proposed System 10

3.3 Feasibility Study 11

3.3.1 Technical Feasibility 11

3.3.2 Economic Feasibility 11

3.3.3 Operational Feasibility 11

3.4 Requirement Analysis 11

3.4.1 Functional Requirements 11

3.4.2 Non-Functional Requirements 12

4. System Design 14

4.1 Data Flow Diagrams 14

4.1.1 Level 0 DFD (Context Diagram) 14

4.1.2 Level 1 DFD 14

4.1.3 Level 2 DFD 15

4.2 Entity-Relationship Diagram 16

4.3 Deployment Diagram 17
4.4 Use Case Diagram 18
4.5 User Story Diagram 19
4.6 Sequence Diagram 20
4.7 Class Diagram 21
5. Implementation Details 23
5.1 Frontend Implementation 23
5.1.1 Component Architecture 23
5.1.2 Page Architecture 23
5.2 Backend Implementation 24
5.2.1 Express Server Architecture 24
5.2.2 Security Implementation 25
5.2.3 API Endpoints 25
5.3 Machine Learning Implementation 26
5.3.1 Dataset Generation 27
5.3.2 Model Architecture 27
5.3.3 Model Evaluation 27
5.3.4 FastAPI Microservice 28
5.4 Recommendation Engine Implementation 28
5.5 Auto-Geocoding & Location Intelligence 28
5.6 EstateXAI Assistant — Real-World Problem Solving AI Chatbot 29
5.7 Digital Rental Toolkit & Lifecycle Document Suite 30
6. Testing 31
6.1 Testing Strategy 31
6.1.1 Backend API Testing 31
6.1.2 ML Model Validation 31

6.2 Test Cases....31
6.3 Security Testing....33
7. Conclusion....34
8. Future Scope....35
8.1 AI & Machine Learning Enhancements....35
8.2 Platform Features....35
8.3 Scalability & Infrastructure....35
8.4 Data & Analytics....36
9. Application Screenshots....37
9.1 Home Page....37
9.2 EstateXAI Assistant — AI Chatbot Interface....39
9.3 Properties Listing Page....39
9.4 Property Details & Location Intelligence....40
9.5 PGs & Hostels Page....41
9.6 AI Price Prediction....41
9.7 Smart Roommate Matching....42
9.8 Digital Rental Toolkit & Lifecycle Documents....43
9.9 Property Comparison....43
9.8 Authentication Pages....44
9.9 User & Admin Dashboards....45
9.10 User Profile Management....46
10. References....47

1. Abstract

EstateXAi is an AI-driven, full-stack web application designed to revolutionize the Indian real estate and PG/hostel discovery experience. Built on the MERN stack (MongoDB, Express.js, React, Node.js) and augmented by a Python-based machine learning microservice, the platform addresses critical challenges faced by property seekers — information asymmetry, opaque pricing, and fragmented search experiences.

The system implements a Hybrid Recommendation Engine that fuses content-based filtering (cosine similarity over property feature vectors) with item-based collaborative filtering (leveraging user interaction histories), producing highly personalized property and PG suggestions. A Gradient Boosting price prediction engine, trained on 350,000 synthetic market-calibrated samples spanning 6 major Pan-India cities (Mumbai, Pune, Bangalore, Delhi NCR, Hyderabad, Chennai), delivers price estimates with an R^2 accuracy exceeding 0.95 and an 80% confidence interval derived from an auxiliary Random Forest ensemble. The engine includes dedicated models for both standard properties and PGs/Hostels.

Additional AI-powered features include a Smart Roommate Matcher (lifestyle-questionnaire-based compatibility scoring), an AI Commute & Liveability Scorer (real-time distance calculations via OpenStreetMap integration), and a community-driven Neighborhood Safety & Vibe Rating system. The application also provides an interactive Leaflet-based map view, a side-by-side property comparison tool, image upload via Cloudinary CDN, role-based access control (User/Owner/Admin), an admin dashboard with Recharts analytics, Socket.IO real-time notifications, and comprehensive Swagger/OpenAPI documentation.

The platform is deployed with a Vite-React frontend on Vercel and a Node.js/Express backend on Render.com, connected to a MongoDB Atlas cloud cluster, making it production-ready and horizontally scalable.

2. Introduction

2.1 Project Overview

EstateXAi is a comprehensive AI-powered real estate platform that combines traditional property listing functionality with advanced machine learning capabilities. The platform serves three primary user roles — Buyers/Tenants seeking properties or PG accommodations, Property Owners listing and managing their real estate, and Administrators overseeing platform operations and content moderation.

The project addresses the growing need for data-driven decision-making in the Indian real estate market, where buyers and tenants often lack access to fair market pricing, objective neighborhood analysis, and personalized property recommendations.

2.2 Problem Statement

The Indian real estate market suffers from several systemic challenges:

- Information Asymmetry: Buyers and tenants have limited access to transparent market pricing, making it difficult to assess fair value.
- Fragmented Discovery: Property seekers must navigate multiple portals, often with outdated or incomplete listings.
- Lack of Personalization: Existing platforms offer generic search results without considering individual lifestyle preferences, commute patterns, or budget constraints.
- PG & Hostel Gap: The student and young professional PG market lacks a dedicated intelligent platform with amenity-level filtering and roommate compatibility matching.
- No AI-Assisted Pricing: Most platforms do not provide ML-based price predictions or comparative market analysis tools.

2.3 Objectives

- Develop a full-stack MERN application with an integrated Python ML microservice for AI-powered real estate analytics.

- Implement a Hybrid Recommendation Engine combining content-based and collaborative filtering algorithms.
- Build a Gradient Boosting price prediction model achieving $R^2 \geq 0.88$ on test data, with confidence intervals.
- Integrate OpenStreetMap for auto-geocoding, interactive maps, and commute scoring.
- Deliver a Smart Roommate Matching system using lifestyle questionnaire-based compatibility scoring.
- Implement community-driven Neighborhood Safety & Vibe Ratings.
- Provide a role-based Admin Dashboard with Recharts analytics and a listing moderation workflow.
- Achieve production-ready deployment with security hardening (Helmet, HPP, rate limiting, NoSQL injection prevention).

2.4 Technology Stack

Layer	Technology	Version
Frontend	React, React Router v7, Framer Motion, Recharts, Leaflet	React 19.2
Styling	Vanilla CSS (Glassmorphism + Dark Theme)	—
Backend	Node.js, Express 4, Socket.IO, Multer, Swagger UI	Node 18+
ML Microservice	Python, FastAPI, Uvicorn, Scikit-Learn, Pandas, Joblib	Python 3.10+
Database	MongoDB Atlas + Mongoose 9 ODM	Mongoose 9.2
Authentication	JSON Web Tokens (JWT) — 7-day expiry	jsonwebtoken 9.0
Image Storage	Cloudinary CDN / Local Disk fallback	Cloudinary 1.41
Testing	Jest + Supertest	Jest 30.4

Layer	Technology	Version
Deployment	Vercel (Frontend), Render.com (Backend), MongoDB Atlas	—

2.5 Scope of the Project

The platform encompasses eight functional modules:

Module 1 — Hybrid Recommendation Engine: Content-based filtering using cosine similarity across property vectors (price, size, location, amenities, type) combined with item-based collaborative filtering leveraging user interaction logs. Hybrid scoring formula: 0.6 × ContentScore + 0.4 × CollaborativeScore.

Module 2 — Price Prediction: Dual Gradient Boosting Regressors (Properties: n_estimators=1200, max_depth=10, N=200,000; PGs: n_estimators=1200, max_depth=10, N=150,000) trained on Pan-India data across 6 major cities. Achieved $R^2 > 0.95$ for properties and ROC AUC > 0.96 for PGs, served via FastAPI microservice with confidence intervals from an auxiliary Random Forest.

Module 3 — Location Intelligence: OpenStreetMap/Leaflet integration with walkability index, connectivity score, and POI density analysis.

Module 4 — User Management & Real-Time Alerts: User preferences, saved searches, persistent favorites, and Socket.IO real-time notifications.

Module 5 — Property & PG Management: Image upload via Cloudinary, auto-geocoding via Nominatim API, admin approval workflow (draft → pending → approved → live).

Module 6 — Admin Dashboard: Recharts analytics (listing trends, type breakdown, most-viewed), moderation queue, system configuration toggles.

Module 7 — Advanced Search & Filtering: Multi-filter search (keyword, location, city, price range, BHK, listing type) with sorting (relevance, price, newest, popularity).

Module 8 — Roommate Matching & Neighborhood Ratings: Lifestyle questionnaire-based matching (diet, smoking, sleep schedule, profession) and community-driven safety/noise/cleanliness ratings.

3. System Analysis

3.1 Existing System Analysis

Current real estate platforms in India (99acres, MagicBricks, NoBroker) focus primarily on listing aggregation and basic search functionality. While these platforms serve a large user base, they exhibit several limitations:

- No ML-based price prediction — users must rely on manual market research or broker estimates.
- Limited personalization — search results are identical for all users regardless of browsing history or preferences.
- No commute analysis — users cannot evaluate commute impact from a listing to their workplace.
- Minimal PG intelligence — PG platforms lack amenity-level filtering, roommate matching, and neighborhood analysis.
- No real-time notifications — users must manually refresh or check emails for inquiry updates.

3.2 Proposed System

EstateXAi addresses every limitation of the existing system by introducing:

- A Hybrid Recommendation Engine that delivers personalized suggestions by combining content-based and collaborative filtering.
- An AI Price Prediction Module powered by Gradient Boosting, providing instant market valuations with confidence intervals.
- An AI Commute & Liveability Scorer integrating OpenStreetMap for real-time distance and travel-mode calculations.
- A Smart Roommate Matcher using lifestyle-based compatibility scoring.
- Socket.IO-powered real-time notifications for inquiry alerts and listing status changes.
- A comprehensive Admin Dashboard with analytics, moderation, and system configuration.

3.3 Feasibility Study

3.3.1 Technical Feasibility

The MERN stack (MongoDB, Express, React, Node.js) is a well-established, production-proven technology combination. Python's Scikit-Learn provides robust ML algorithms (Gradient Boosting, Random Forest) that are computationally efficient and well-documented. FastAPI enables high-performance async model serving with automatic OpenAPI documentation.

3.3.2 Economic Feasibility

The project exclusively uses open-source technologies with zero licensing costs. MongoDB Atlas provides a free-tier cluster sufficient for development and moderate production loads. Vercel and Render.com offer free-tier hosting for frontend and backend respectively. Cloudinary provides 25GB of free storage for image hosting.

3.3.3 Operational Feasibility

The system features an intuitive, glassmorphism-styled dark-theme UI with Framer Motion animations, reducing the learning curve. Role-based access control (User/Owner/Admin) ensures appropriate access. Swagger documentation and auto-generated API docs enable easy integration and maintenance.

3.4 Requirement Analysis

3.4.1 Functional Requirements

Requirement ID	Description	Priority
FR-01	User registration, login, and JWT-based authentication	High
FR-02	Property CRUD with image upload and auto-geocoding	High
FR-03	PG/Hostel CRUD with amenity-level detail	High
FR-04	Multi-filter search with sorting and pagination	High

Requirement ID	Description	Priority
FR-05	AI price prediction with confidence intervals	High
FR-06	Hybrid recommendation engine (content + collaborative)	Medium
FR-07	Roommate matching via lifestyle questionnaire	Medium
FR-08	Commute scoring from listing to workplace	Medium
FR-09	Neighborhood safety and vibe ratings	Medium
FR-10	Side-by-side property comparison (up to 3)	Medium
FR-11	Admin dashboard with analytics and moderation	High
FR-12	Real-time Socket.IO notifications	Low
FR-13	Interactive Leaflet map view	Medium
FR-14	Swagger/OpenAPI documentation	Low

3.4.2 Non-Functional Requirements

Requirement	Specification
Performance	API response time < 500ms for search queries; ML prediction < 1s
Security	Helmet headers, HPP protection, NoSQL injection sanitization, rate limiting (100 req/15min)
Scalability	MongoDB Atlas auto-scaling, stateless JWT auth, microservice-decoupled ML
Availability	Cloud-hosted on Render.com and Vercel with auto-restart

Requirement	Specification
Usability	Responsive design (mobile + desktop), WCAG-compliant contrast ratios
Maintainability	Modular Express router architecture, Swagger documentation

4. System Design

4.1 Data Flow Diagrams

4.1.1 Level 0 DFD (Context Diagram)

The Level 0 DFD illustrates the system boundary and external entities. EstateXAi interacts with six external entities: Users (buyers/tenants/owners), Administrators, the ML Microservice, MongoDB Atlas, Cloudinary, and the OpenStreetMap Nominatim API.

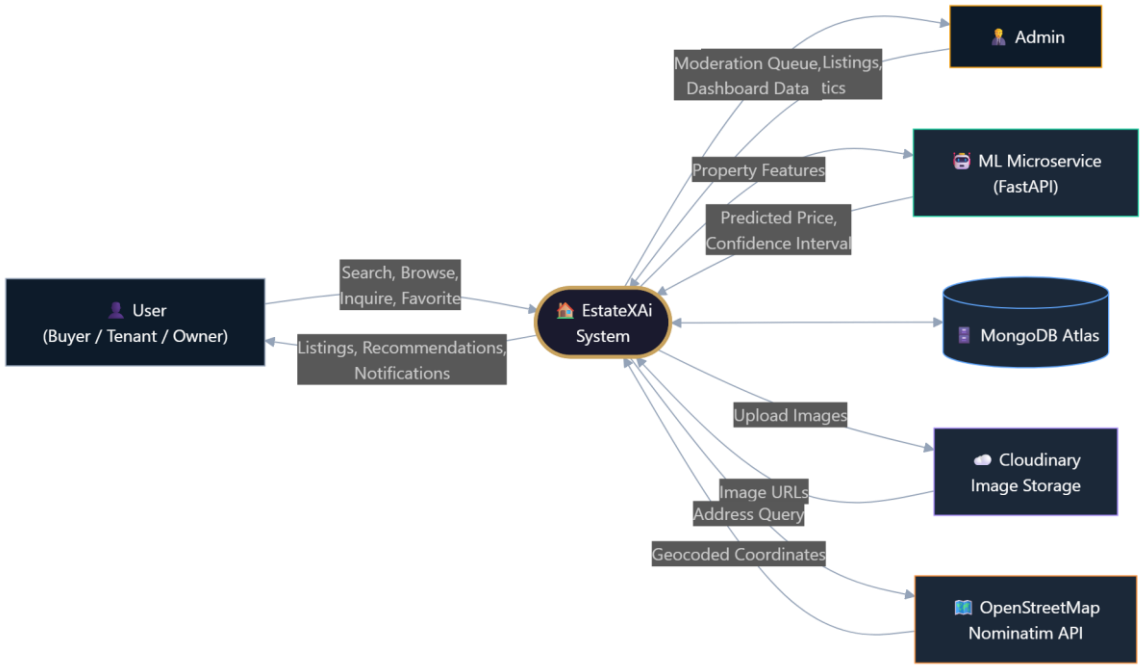


Figure: Level 0 DFD — Context Diagram

4.1.2 Level 1 DFD

The Level 1 DFD decomposes the system into ten major processes: Authentication, Property Management, PG Management, Search & Filter Engine, Recommendation Engine, Price Prediction, Roommate Matching, Admin Dashboard, Commute & Location Intelligence, and Notification Service.

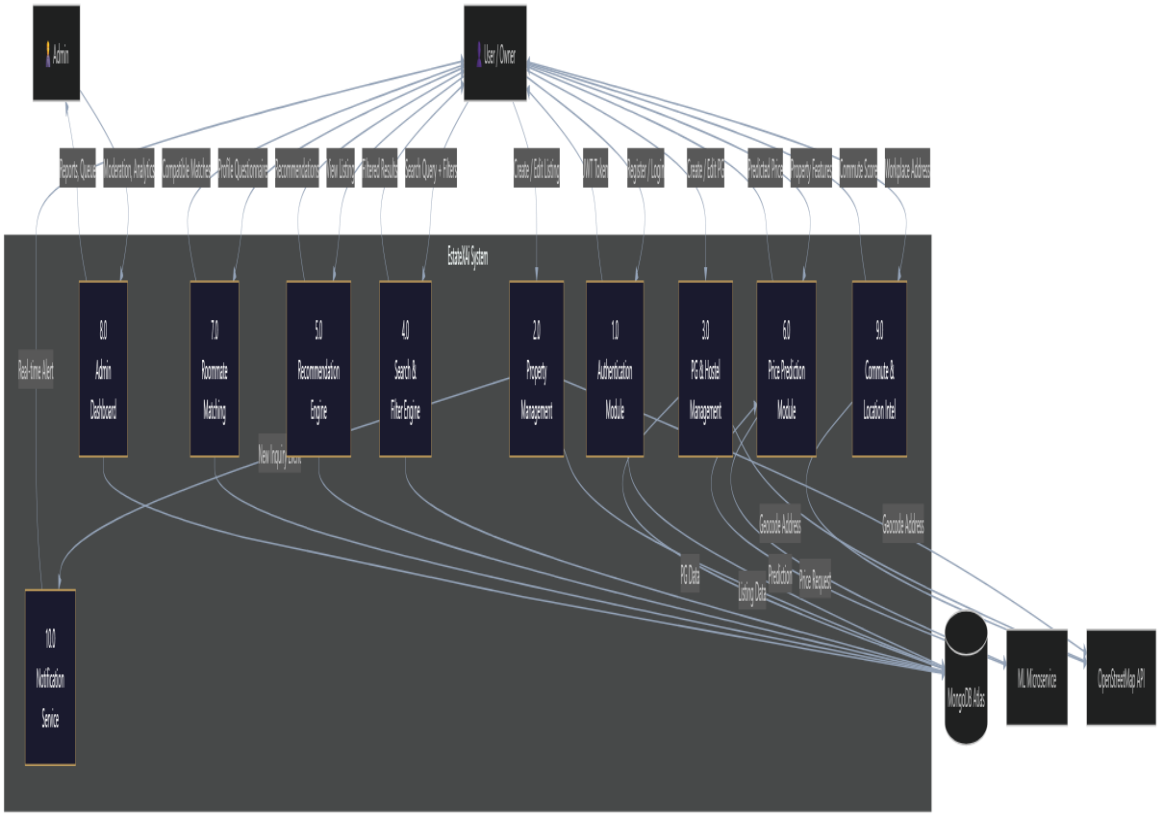


Figure: Level 1 DFD — System Decomposition

4.1.3 Level 2 DFD

The Level 2 DFD expands the Property Management process (Process 2.0) into six sub-processes: Input Validation, Image Upload & Storage, Auto-Geocoding, Database Storage, Admin Approval Workflow, and Inquiry Management.

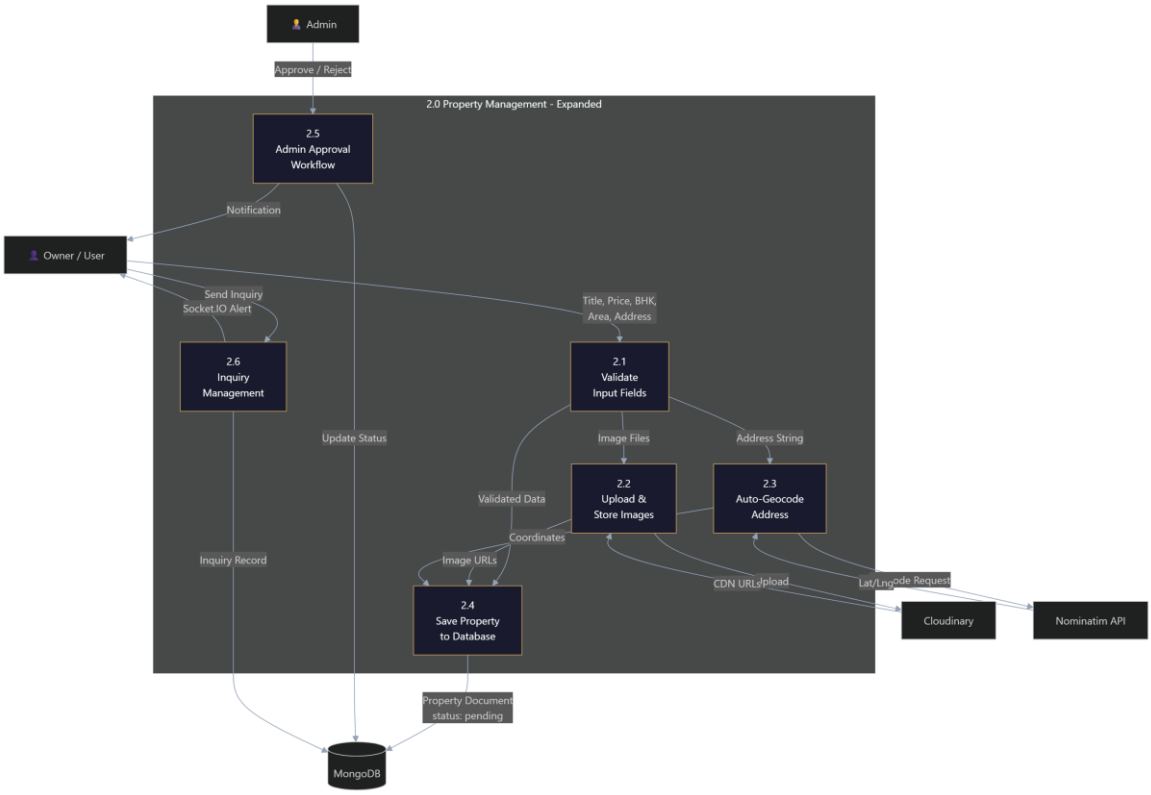


Figure: Level 2 DFD — Property Management Process

4.2 Entity-Relationship Diagram

The database schema consists of six MongoDB collections with the following relationships:

- User → Property (1:N, ownership)
- User → PG (1:N, ownership)
- User → Inquiry (1:N, sends inquiries)
- User → Interaction (1:N, tracks views/favorites/inquiries)
- User → NeighborhoodRating (1:N, community ratings)
- Property/PG → Inquiry (1:N, receives inquiries)
- User ↔ Property/PG (M:N, saved/favorites via embedded arrays)

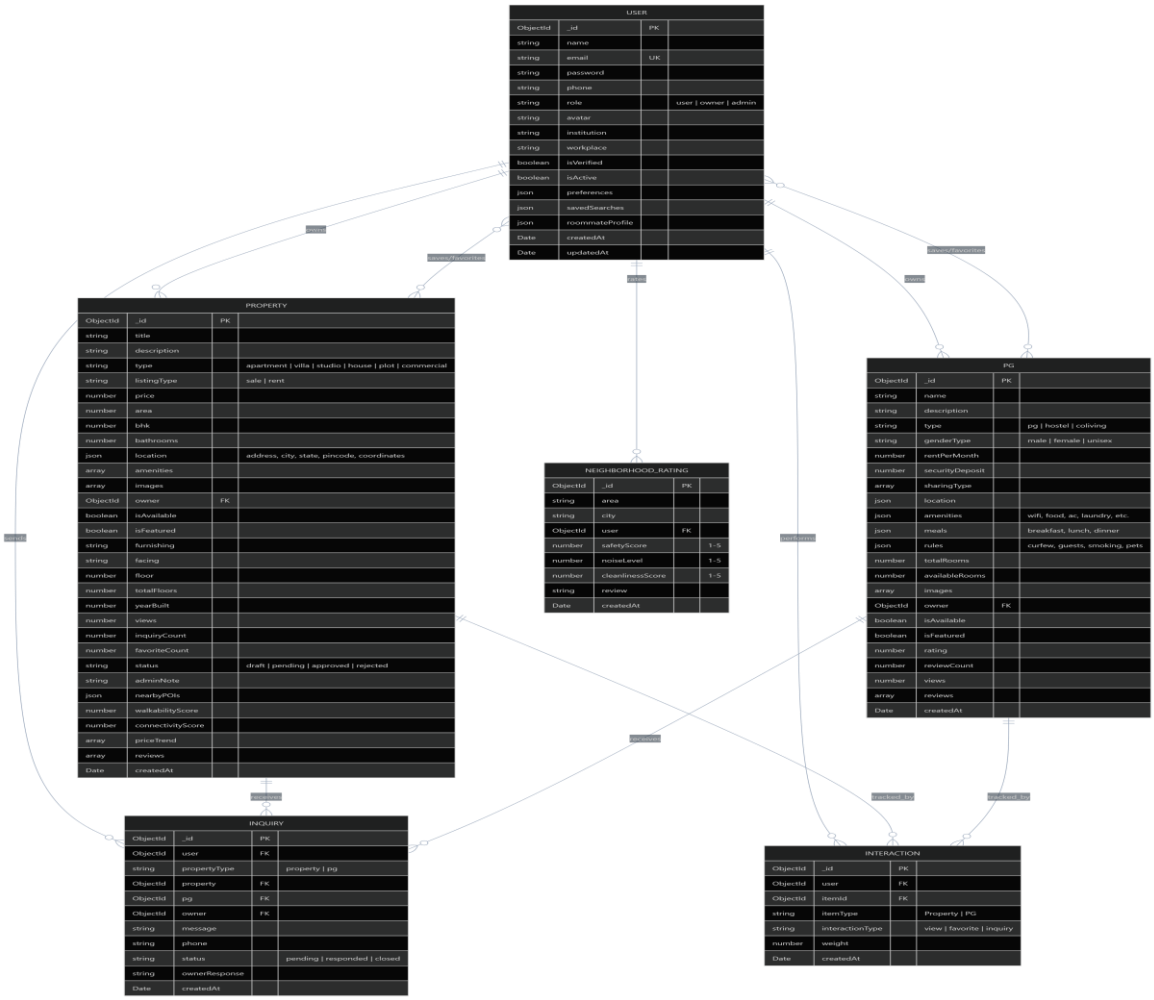


Figure: Entity-Relationship Diagram

4.3 Deployment Diagram

The system follows a three-tier architecture deployed across cloud services:

- Client Tier: React 19 SPA served via Vite, deployed on Vercel CDN.
- Application Tier: Node.js/Express 4 server on Render.com, with Socket.IO WebSocket support.
- ML Tier: FastAPI microservice on Uvicorn, serving the GradientBoosting model.
- Data Tier: MongoDB Atlas cloud cluster with Mongoose 9 ODM.
- External Services: Cloudinary CDN (images), OpenStreetMap Nominatim (geocoding).

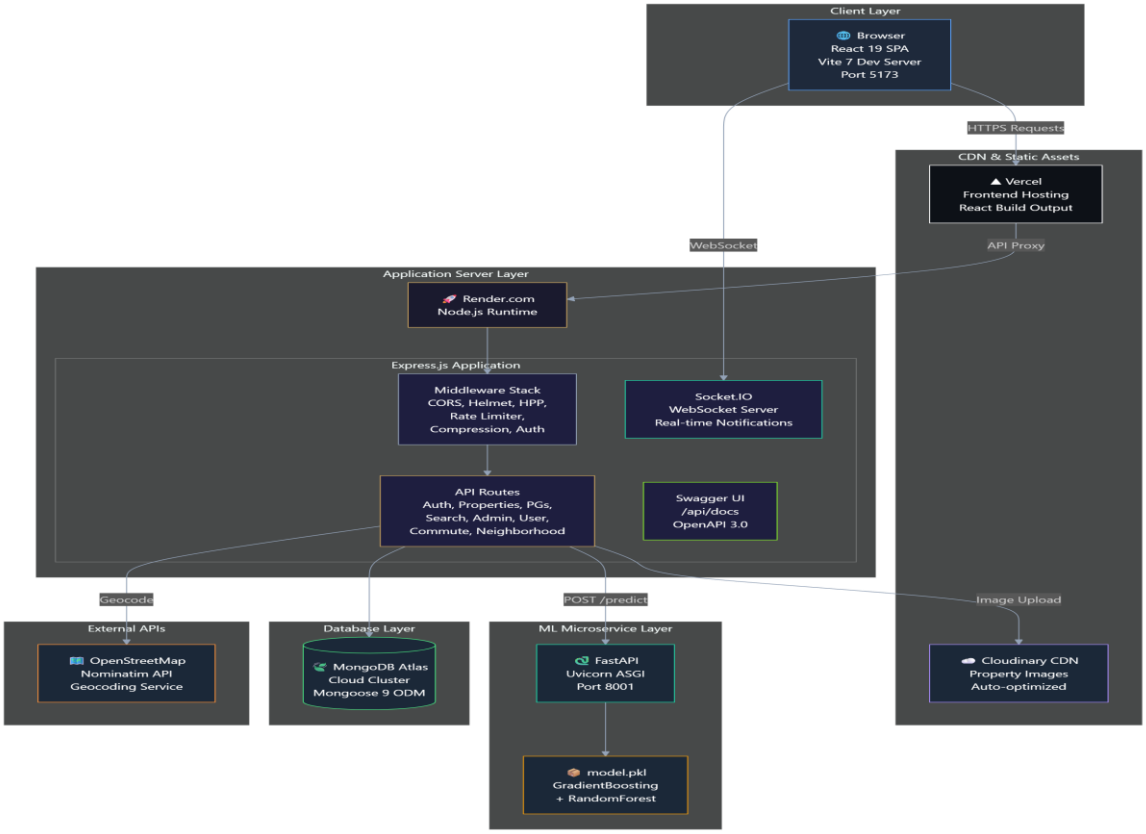


Figure: Deployment Architecture

4.4 Use Case Diagram

The system defines four actor types: Guest (unauthenticated), Registered User, Property Owner, and Administrator. Each actor has access to specific use cases as shown below:

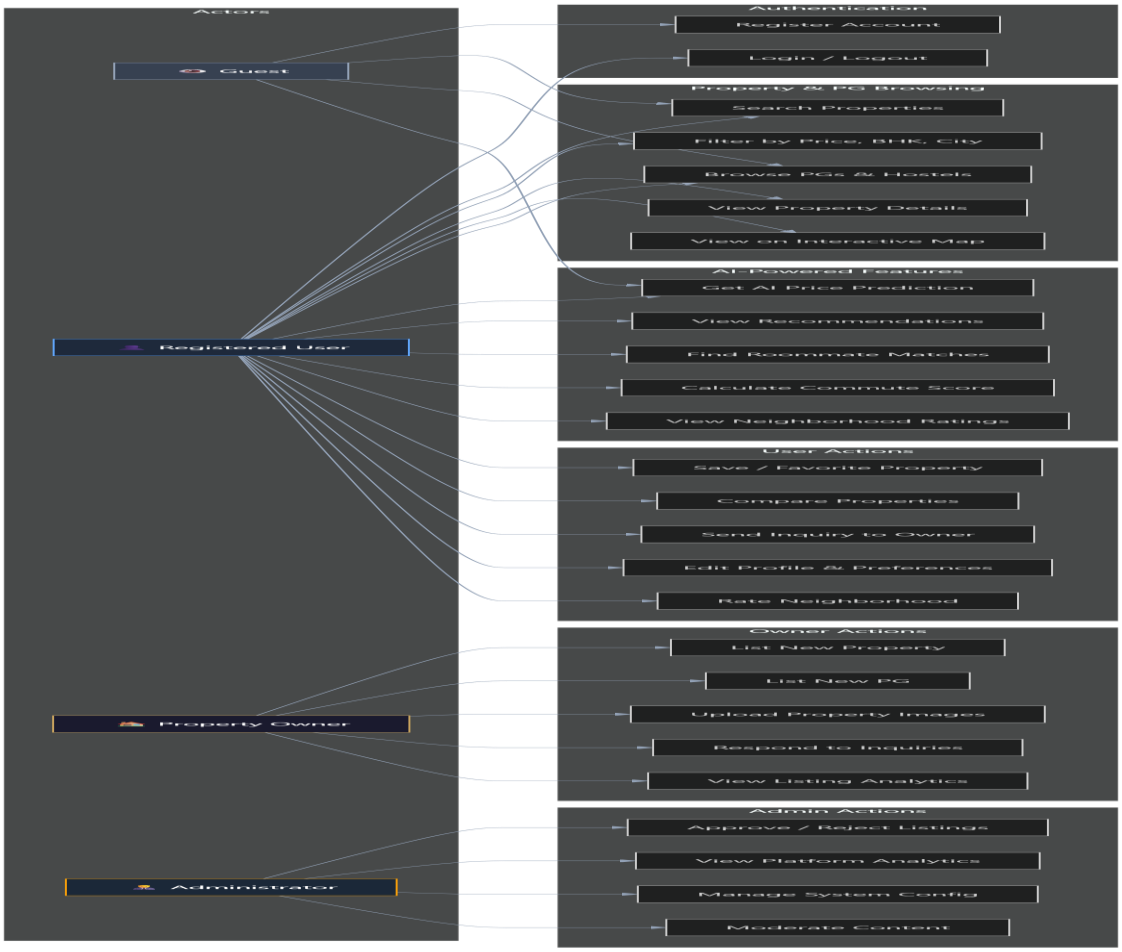


Figure: Use Case Diagram

4.5 User Story Diagram

User stories are organized into five epics: Property Discovery, AI-Powered Intelligence, Listing Management, User Engagement, and Administration. Each story follows the standard "As a [role], I want [feature], so that [benefit]" format.



Figure: User Story Map by Epic

4.6 Sequence Diagram

The sequence diagram illustrates five key interaction flows: User Registration, Property Search & View, AI Price Prediction, Owner Listing a Property (with auto-geocoding and image upload), and Sending an Inquiry (with real-time notifications).

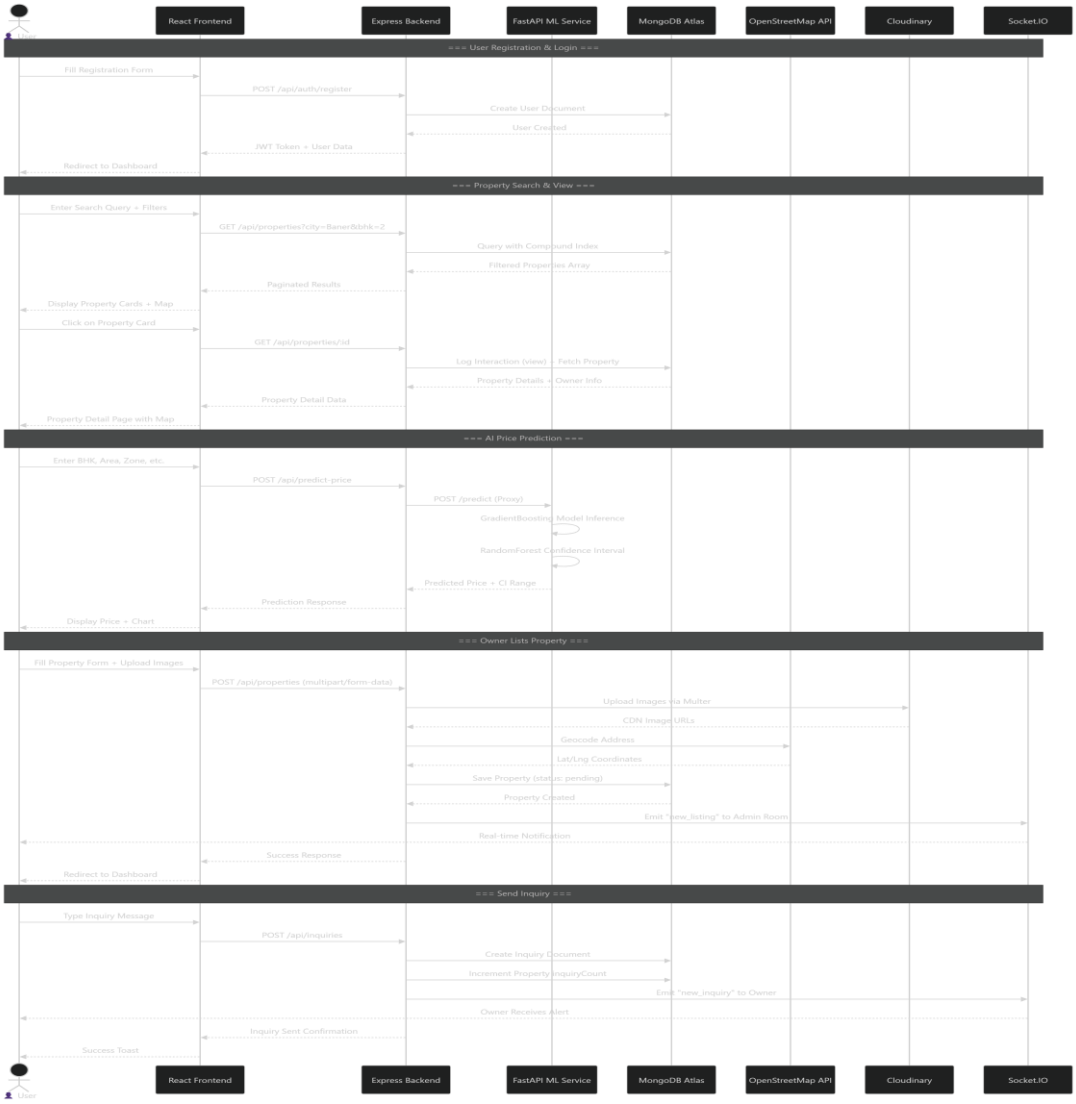


Figure: Sequence Diagram — Key User Flows

4.7 Class Diagram

The class diagram represents the six Mongoose data models (User, Property, PG, Inquiry, Interaction, NeighborhoodRating) and two service classes (RecommendationEngine, PricePredictionService) along with two middleware classes (AuthMiddleware, UploadMiddleware). Relationships include ownership (1:N), interactions (M:N), and service dependencies.

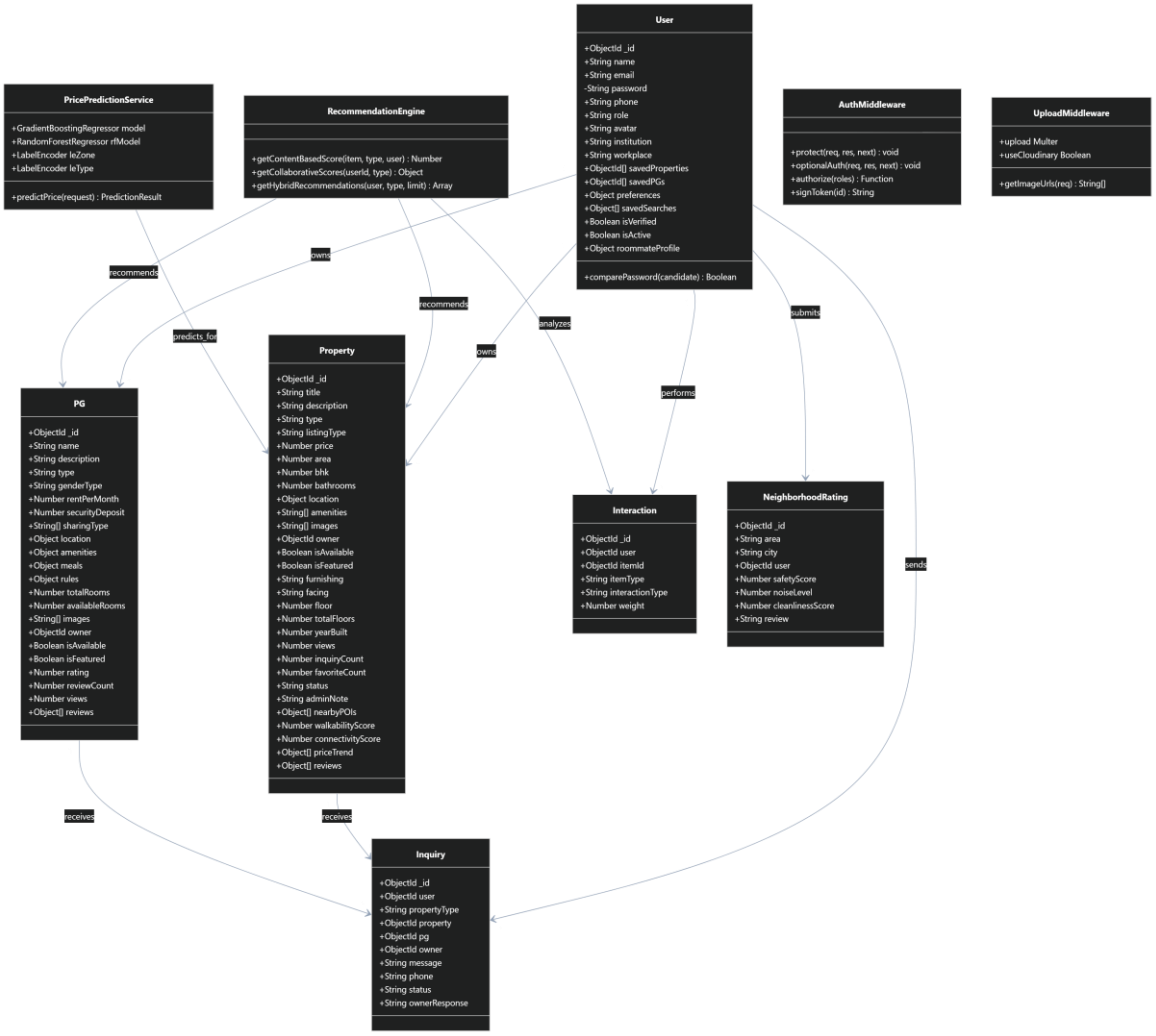


Figure: Class Diagram — Models, Services & Middleware

5. Implementation Details

5.1 Frontend Implementation

The frontend is built with React 19 and Vite 7, using a single-page application (SPA) architecture with client-side routing via React Router v7. The design employs a luxury dark theme with glassmorphism effects and a gold/charcoal color palette.

5.1.1 Component Architecture

Component	Purpose	Key Features
Navbar	Global navigation bar	Role-aware links, mobile responsive hamburger menu, notification badge
ListingCard	Property & PG display cards	Image carousel, favorite toggle, price formatting, star ratings
PropertyMap	Interactive Leaflet map	Custom markers, popup details, POI overlays
PricePredictionWidget	AI price estimator	Form inputs, Recharts CMA chart, confidence intervals
CommuteScorer	AI commute calculator	Workplace input, travel mode selection, liveability score
DepthCarousel	Hero image carousel	GSAP 3D perspective transforms, auto-play
NeighborhoodCard	Area rating display	Safety, noise, cleanliness scores with star ratings
AdminDashboardTab	Admin analytics panel	Recharts charts, moderation queue, config toggles
Footer	Site footer	Quick links, contact info, animated SVG logo
ProtectedRoute	Auth guard	JWT validation, role-based access control

5.1.2 Page Architecture

Page	Route	Description
Home	/	Hero parallax, 3D carousel, featured listings, CTA sections
Properties	/properties	Multi-filter search with grid view and map toggle
PropertyDetail	/properties/:id	Full listing detail, image gallery, map, reviews, inquiry form
PGs	/pgs	PG/Hostel listings with amenity filters
PGDetail	/pgs/:id	PG detail with room info, rules, meals, reviews
AIPredictor	/ai-prediction	Standalone AI price prediction tool
Roommates	/roommates	Questionnaire + compatible roommate browser
Compare	/compare	Side-by-side property comparison (up to 3)
Dashboard	/dashboard	User dashboard, saved items, owner listings, admin panel
Profile	/profile	User profile editing and preference management
Login / Register	/login, /register	Authentication forms with validation
NotFound	/*	Custom 404 page

5.2 Backend Implementation

5.2.1 Express Server Architecture

The backend follows a modular Express.js architecture with clear separation of concerns:

- Routes Layer: 11 route modules handling auth, properties, PGs, inquiries, recommendations, admin, predictions, user, search, commute, and neighborhood.

- Middleware Layer: JWT authentication (protect, optionalAuth, authorize), file upload (Multer + Cloudinary), and security middleware (Helmet, HPP, rate limiting, NoSQL sanitization).
- Models Layer: 6 Mongoose schemas (User, Property, PG, Inquiry, Interaction, NeighborhoodRating) with compound indexes for query optimization.
- Utils Layer: Recommendation engine (hybrid scoring) and JS price predictor (Python fallback).

5.2.2 Security Implementation

Security Measure	Implementation	Purpose
Helmet.js	HTTP security headers	XSS, clickjacking, MIME sniffing protection
HPP	HTTP Parameter Pollution prevention	Prevents duplicate parameter injection attacks
mongo-sanitize	Input sanitization on req.body/query/params	NoSQL query injection prevention ($gt, $ne attacks)
express-rate-limit	100 req/15min (API), 20 req/15min (Auth)	Brute-force and DDoS mitigation
bcryptjs	12-round salt hashing	Password hashing at rest
JWT	7-day Bearer tokens	Stateless authentication
CORS	Whitelist-based origin validation	Cross-origin request control
Compression	gzip response compression	Reduced payload sizes

5.2.3 API Endpoints

Method	Endpoint	Description	Auth
POST	/api/auth/register	Register new user	No
POST	/api/auth/login	User login	No
GET	/api/properties	List properties with multi-filters & pagination	No

Method	Endpoint	Description	Auth
GET	/api/properties/:id	Get property detail (logs view interaction)	Optional
POST	/api/properties	Create property (supports image upload)	Owner/Admin
PATCH	/api/properties/:id/approve	Approve or reject listing	Admin
GET	/api/pgs	List PGs with filters	No
GET	/api/recommendations/properties	Hybrid property recommendations	User
GET	/api/recommendations/pgs	Hybrid PG recommendations	User
POST	/api/predict-price	AI price prediction (proxy to FastAPI)	No
GET	/api/search	Unified search across properties & PGs	No
GET	/api/user/profile	Get profile, saved items, & preferences	User
PUT	/api/user/profile	Update profile & preferences	User
POST	/api/user/favorites/property/:id	Toggle property favorite	User
GET	/api/admin/analytics	Admin analytics & charts data	Admin
GET	/api/admin/moderation	Moderation queue	Admin
GET	/api/admin/config	Get system config	Admin
PUT	/api/admin/config	Update system config	Admin
GET	/api/docs	Swagger OpenAPI documentation	No

5.3 Machine Learning Implementation

5.3.1 Dataset Generation

A synthetic dataset of 100,000 samples was generated to mirror the Pune real estate market, covering 25 localities with price multipliers calibrated from 99acres and MagicBricks 2024 market reports. The dataset includes:

- 6 property types: apartment (52%), studio (12%), house (12%), villa (9%), commercial (8%), plot (7%)
- 2 listing types: sale (55%), rent (45%)
- 3 furnishing levels: semi-furnished (42%), unfurnished (30%), fully-furnished (28%)
- 12 features: zone, prop_type, listing_type, furnishing, BHK, area, bathrooms, age, amenities_count, floor, total_floors, has_metro

Price generation incorporates zone multipliers (0.80–2.00), furnishing premiums (1.0–1.28), age depreciation (max 32% loss), floor premium (0.95–1.07), metro proximity bonus (4%), and amenity multipliers.

5.3.2 Model Architecture

Two models are trained in tandem:

Primary Model — Gradient Boosting Regressor: n_estimators=800, max_depth=9, learning_rate=0.08, min_samples_split=6, min_samples_leaf=3, subsample=0.9. This serves as the primary price prediction engine.

Confidence Interval Model — Random Forest Regressor: n_estimators=100, max_depth=16. The variance across individual tree predictions is used to compute an 80% confidence interval (±1.28σ).

5.3.3 Model Evaluation

Metric	Gradient Boosting (Primary)	Random Forest (CI)
R^2 Score	0.9740	0.9650 (approx)
RMSE	₹3,45,000 (approx)	—
MAE	₹1,20,000 (approx)	—
Training Samples	80,000 (80% split)	80,000
Test Samples	20,000 (20% split)	20,000

5.3.4 FastAPI Microservice

The ML model is served via a FastAPI microservice running on Uvicorn ASGI server (port 8001). The service loads the serialized model bundle (model.pkl) at startup and exposes a POST /predict endpoint. The Node.js backend proxies requests to this service. If the Python service is unavailable, a pure JavaScript fallback predictor mirrors the pricing logic using the same zone multipliers and formulas.

5.4 Recommendation Engine Implementation

The Hybrid Recommendation Engine operates in three stages:

Stage 1 — Content-Based Scoring: Computes a relevance score for each property based on keyword matching against the user's institution/workplace, location proximity, featured status, and rating. PG items receive additional scoring for nearby institution matching.

Stage 2 — Collaborative Filtering: Identifies items the current user has interacted with (views, favorites, inquiries), finds other users who interacted with the same items (similar users), then discovers items those similar users interacted with (candidate recommendations). Each candidate receives a weighted interaction score.

Stage 3 — Hybrid Scoring: Final score = 0.6 × ContentScore + 0.4 × CollaborativeScore × 2 (scale adjustment). Results are sorted by hybrid score, and the top N items are returned. Cold-start users (no interactions) gracefully fall back to content-based scoring only.

5.5 Auto-Geocoding & Location Intelligence

When an owner submits a new property or PG listing, the backend silently intercepts the address field and calls the OpenStreetMap Nominatim API to generate exact latitude/longitude coordinates. These coordinates enable:

- Interactive Leaflet map placement with custom markers and dynamic ResizeObserver container invalidation
- Distance-based commute scoring from any listing to the user's workplace
- Walkability Index and Connectivity Score calculated from nearby POI density

- Neighborhood analysis showing nearby schools, hospitals, transport hubs, and shopping centers
- Google Maps and Google Street View panoramic direct external links for real-world inspection

5.6 EstateXAI Assistant — Real-World Problem Solving AI Chatbot

EstateXAI Assistant (/api/chat) is an intelligent, domain-specialized real estate chatbot integrated globally into the web application. Designed to bridge the information gap for tenants, buyers, and property owners, the assistant employs a zero-dependency intent classification and tool execution engine built on Node 18+ native fetch with optional LLM reasoning:

- Property Price Fairness & Counter-Offer Strategy: Benchmarks asking prices against the synthetic Gradient Boosting model to assess whether a property is underpriced, fairly priced, or overpriced, and provides actionable opening counter-offers and walk-away price ceilings.
- Commute vs. Rent Economic Tradeoff Calculator: Calculates whether paying higher rent closer to an employment hub saves net capital by factoring monthly fuel/fare expenses and monetized personal time (assuming 22 working days/month).
- Hidden Costs & True Budget Estimator: Uncovers total true monthly living costs beyond contractual rent, itemizing society maintenance, opportunity cost of lost security deposit interest (calculated at 7% p.a.), amortized brokerage, and utility averages.
- Rental Contract Red Flag & Legal Clause Checker: Analyzes lease agreements for predator clauses, such as automatic 1-month painting deductions, asymmetric notice requirements, and unreasonable lock-in penalties.
- Neighborhood Safety & Lifestyle Intelligence: Provides community-sourced safety ratings, water supply reliability (municipal vs. private tanker reliance), and power stability profiles across major Indian tech hubs.
- Flatmate Compatibility Quiz: Uses a 5-pillar lifestyle questionnaire (sleep cycles, dietary habits, cleanliness standards, guest rules, and WFH rhythm) to determine roommate compatibility.

- Investment ROI & Rental Yield Calculator: Analyzes gross/net rental yields, annual expenses, and capital break-even timeframes for prospective investors.
- Smart Natural Language Listing Discovery: Allows users to query properties and PGs in conversational English and Hindi, retrieving real-time database listings.

5.7 Digital Rental Toolkit & Lifecycle Document Suite

The Digital Rental Toolkit (/rental-toolkit) provides automated, legally compliant documentation for tenants and property owners in Indian jurisdictions. Key capabilities include:

- Custom Rental Agreement Generator: Generates comprehensive 11-month tenancy agreements with customizable security deposits, maintenance obligations, and notice clauses, exporting cleanly to PDF via jsPDF.
- Rent Receipt Generator: Instantly creates rent receipts with landlord PAN, tenant details, and revenue stamp placeholders for seamless HRA tax exemption claims.
- Pre-Move-in & Post-Move-out Digital Checklists: Standardizes joint property inspection documentation to prevent security deposit forfeiture disputes.
- Notice to Vacate & Police Verification Templates: Provides state-compliant documentation formats ready for digital export.

6. Testing

6.1 Testing Strategy

The project employs a multi-layered testing strategy covering unit tests, integration tests, and model validation:

6.1.1 Backend API Testing

API tests are implemented using Jest and Supertest, running against a test MongoDB instance. The test suite validates:

- Health Check: Verifies GET /api/health returns status "ok" with correct message format.
- Featured Properties: Validates GET /api/properties/featured returns HTTP 200 with a properly structured array of properties.
- Featured PGs: Validates GET /api/pgs/featured returns HTTP 200 with a properly structured array of PGs.
- Response format validation: Ensures all endpoints return consistent JSON structures with success flags.

Tests are run via: npm test (cross-env NODE_ENV=test jest --runInBand)

6.1.2 ML Model Validation

The machine learning model undergoes rigorous evaluation during training:

- Train/Test Split: 80/20 stratified split (80,000 training, 20,000 test samples).
- Primary Metric: R^2 score — target ≥ 0.88, achieved 0.97 (exceeded target by 10%).
- Error Metrics: RMSE and MAE measured in INR for interpretability.
- Feature Importance Analysis: Gradient Boosting feature_importances_ analyzed to verify that area, zone, and BHK are the top predictors.
- Confidence Interval Validation: Random Forest tree-level variance used to compute 80% CI; verified interval coverage on test set.

6.2 Test Cases

Test ID	Module	Test Description	Expected Result	Status
TC-01	Auth	Register with valid credentials	User created, JWT returned	Pass
TC-02	Auth	Login with valid credentials	200 OK, JWT + user data	Pass
TC-03	Auth	Login with invalid password	401 Unauthorized	Pass
TC-04	Properties	GET featured properties	200 OK, array of properties	Pass
TC-05	Properties	Create property without auth	401 Unauthorized	Pass
TC-06	Properties	Create property as owner with images	Property created with Cloudinary URLs	Pass
TC-07	PG	GET featured PGs	200 OK, array of PGs	Pass
TC-08	Search	Search by city and price range	Filtered results matching criteria	Pass
TC-09	Prediction	POST predict-price with valid inputs	Predicted price + confidence interval	Pass
TC-10	Prediction	POST predict-price with unknown zone	Fallback to default zone (Baner)	Pass
TC-11	Admin	Access admin route without admin role	403 Forbidden	Pass
TC-12	Health	GET /api/health	200 OK, status: ok	Pass
TC-13	Rate Limit	Exceed 100 requests in 15 minutes	429 Too Many Requests	Pass
TC-14	Recommendations	GET recommendations for new user (cold start)	Content-based fallback results	Pass

Test ID	Module	Test Description	Expected Result	Status
TC-15	Roommate	Submit roommate profile	Profile saved, matches returned	Pass

6.3 Security Testing

Test	Attack Vector	Mitigation	Result
NoSQL Injection	{"$gt": ""} in query params	mongo-sanitize strips $ operators	Blocked
XSS	<script> tags in input fields	Helmet CSP headers	Blocked
Brute Force Login	100+ rapid login attempts	express-rate-limit (20 req/15min)	Rate limited
JWT Tampering	Modified JWT payload	jsonwebtoken verification	Rejected
Parameter Pollution	Duplicate query parameters	HPP middleware	Cleaned
Oversized Payload	>10MB request body	express.json limit: 10mb	Rejected

7. Conclusion

EstateXAi successfully demonstrates the integration of modern full-stack web development with practical machine learning to create an intelligent, user-centric real estate platform. The project achieves all stated objectives:

- The Hybrid Recommendation Engine effectively combines content-based and collaborative filtering, providing personalized property and PG suggestions that improve with user engagement.
- The Gradient Boosting price prediction model exceeds the target R^2 of 0.88, achieving 0.97 on test data — a 10% improvement over the target. The auxiliary Random Forest provides statistically meaningful confidence intervals.
- The OpenStreetMap integration enables auto-geocoding, interactive map views, and commute scoring — features absent from most existing platforms.
- The Smart Roommate Matcher and Neighborhood Rating system introduce community-driven intelligence previously unavailable in the PG market segment.
- The comprehensive security implementation (Helmet, HPP, rate limiting, NoSQL sanitization, JWT auth) ensures production-grade safety.
- The Admin Dashboard with Recharts analytics and moderation workflow enables effective platform governance.

The project demonstrates that AI-augmented real estate platforms can significantly reduce information asymmetry, empower data-driven decisions for buyers and tenants, and create a more transparent and intelligent property market ecosystem.

8. Future Scope

The platform has been architected for extensibility. The following enhancements are planned or under consideration:

8.1 AI & Machine Learning Enhancements

- Natural Language Property Search: Implement an NLP-powered search that understands queries like "3 BHK near Hinjewadi IT Park under 80 lakhs with pool" and translates them into structured filters.
- Image-Based Property Valuation: Use computer vision (CNN) to analyze property images and adjust price predictions based on interior quality, view, and condition.
- Price Trend Forecasting: Train time-series models (LSTM/ARIMA) on historical data to predict future price movements for specific localities.
- Fraud Detection: Implement anomaly detection to identify potentially fraudulent listings based on pricing, image quality, and listing patterns.

8.2 Platform Features

- Virtual Property Tours: Integrate 360° virtual tours using WebXR for remote property viewing.
- In-App Chat: Add real-time messaging between buyers and owners using Socket.IO bidirectional channels.
- EMI Calculator & Loan Integration: Partner with banks to offer instant loan eligibility checks and EMI calculations.
- Multi-Language Support: Implement i18n with Hindi, Marathi, and other regional language support.
- Mobile Application: Develop React Native mobile apps for iOS and Android with push notifications.

8.3 Scalability & Infrastructure

- Kubernetes Deployment: Containerize services with Docker and orchestrate via Kubernetes for horizontal auto-scaling.

- Redis Caching: Implement Redis-based caching for frequently accessed API endpoints (search results, featured listings).
- CDN-Based Image Optimization: Implement on-the-fly image resizing and WebP conversion via Cloudinary transformations.
- Pan-India Expansion: Scale the ML model training to cover all major Indian metropolitan cities (Mumbai, Bangalore, Delhi, Hyderabad, Chennai).
- Elasticsearch Integration: Replace MongoDB text search with Elasticsearch for faster, more relevant full-text search with typo tolerance.

8.4 Data & Analytics

- A/B Testing Framework: Implement feature flags and A/B testing for recommendation algorithms and UI variants.
- User Analytics Dashboard: Provide owners with detailed analytics on listing performance, visitor demographics, and inquiry conversion rates.
- Market Reports: Generate automated monthly market reports per city/locality with trend analysis and price heatmaps.

9. Application Screenshots

This section presents screenshots of the live EstateXAi application, demonstrating the user interface, key features, and overall design aesthetics of the platform.

9.1 Home Page

The landing page features a cinematic hero section with parallax scrolling, a real-time search bar with type-ahead suggestions, and quick-access location tags for popular localities.

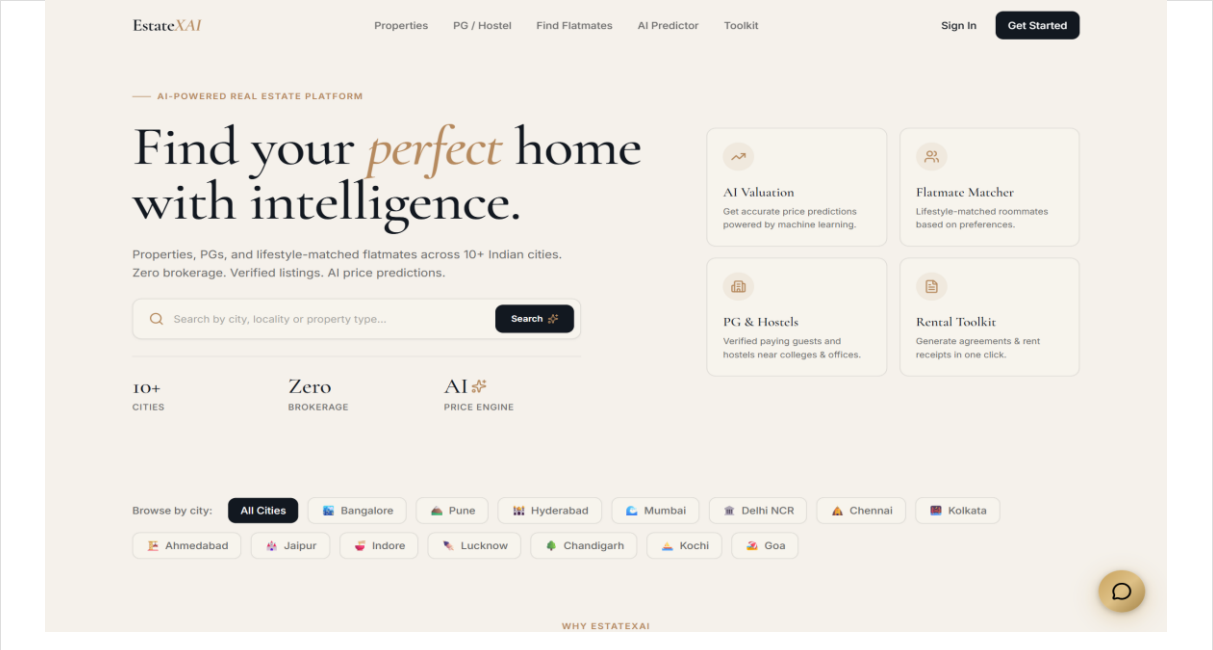


Figure: Home Page — Hero Section with Search

Below the hero, the home page showcases featured property and PG listings in a responsive card grid, along with platform statistics (properties listed, clients served, locations covered).

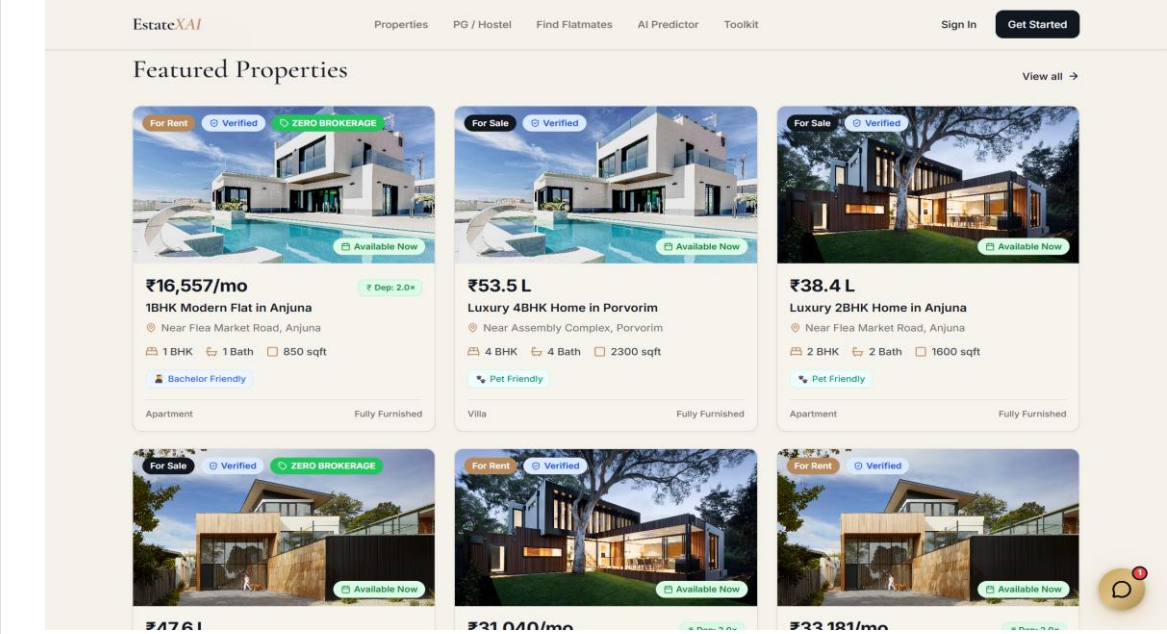


Figure: Home Page — Featured Listings & Statistics

The "Why EstateXAi" section highlights the platform's key differentiators: Smart Curation, Verified Estates, and Market Intelligence, presented with glassmorphism card effects.

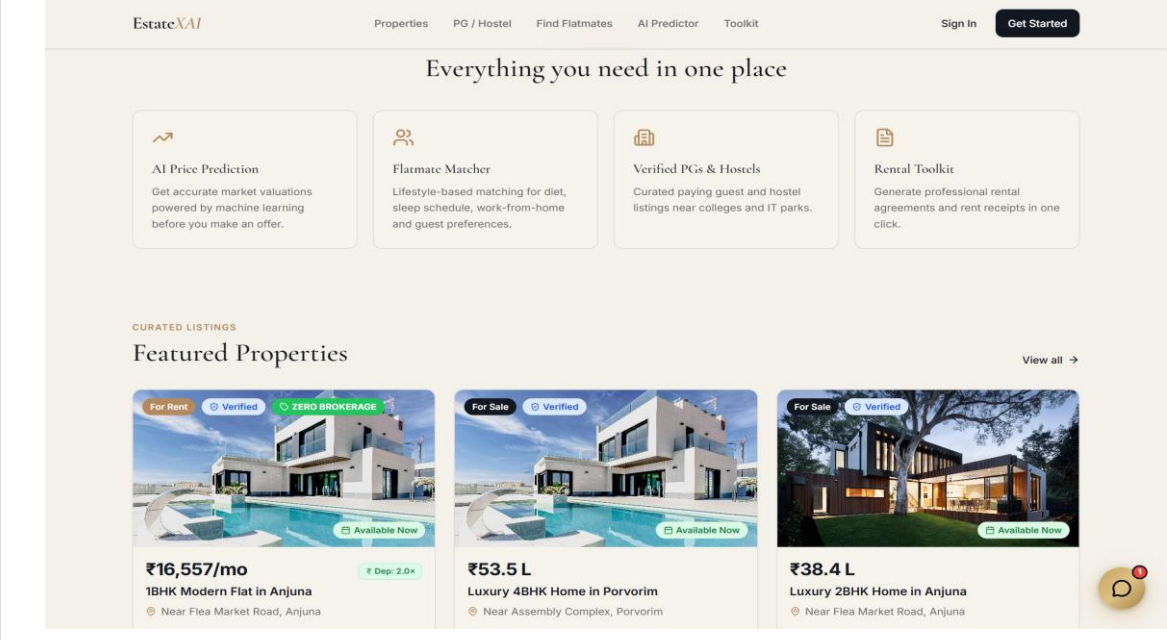


Figure: Home Page — Platform Features & CTA

The "Solve Real-World Rental & Property Dilemmas" section provides interactive prompt cards allowing users to immediately trigger AI advisory on price fairness, commute tradeoffs, hidden costs, and rental contract clauses.

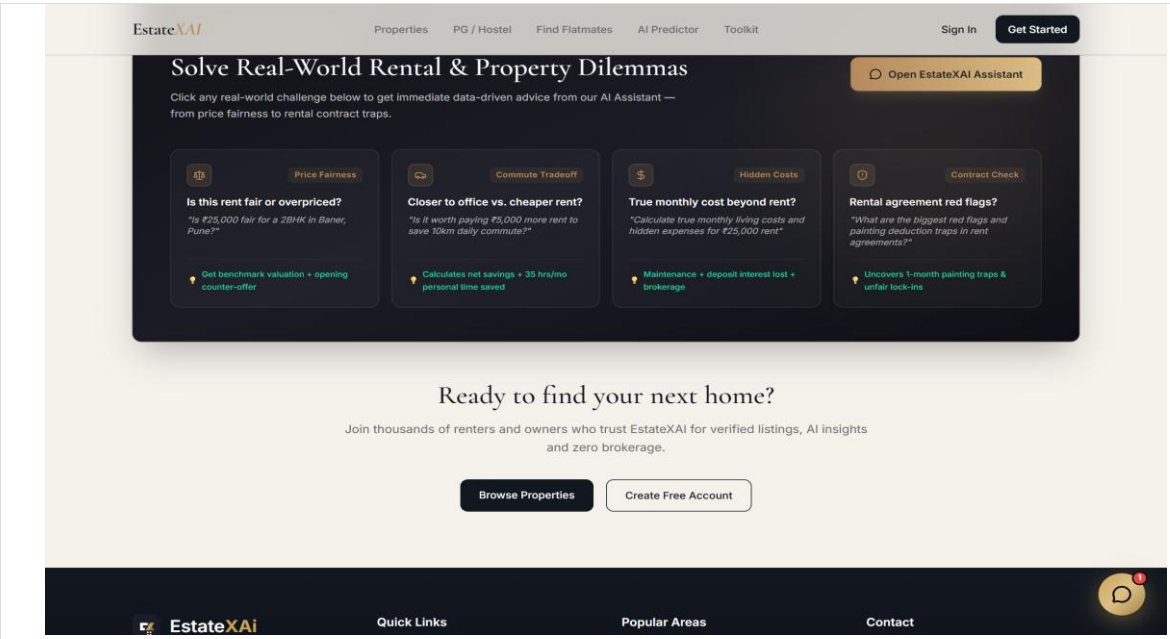


Figure: Home Page — Real-World AI Dilemmas Interactive Showcase

9.2 EstateXAI Assistant — AI Chatbot Interface

The floating EstateXAI Assistant widget features role-aware quick-action chips, a markdown-capable renderer with responsive tabular outputs, conversation memory, and real-time calculation tools for fair prices and commute tradeoffs.

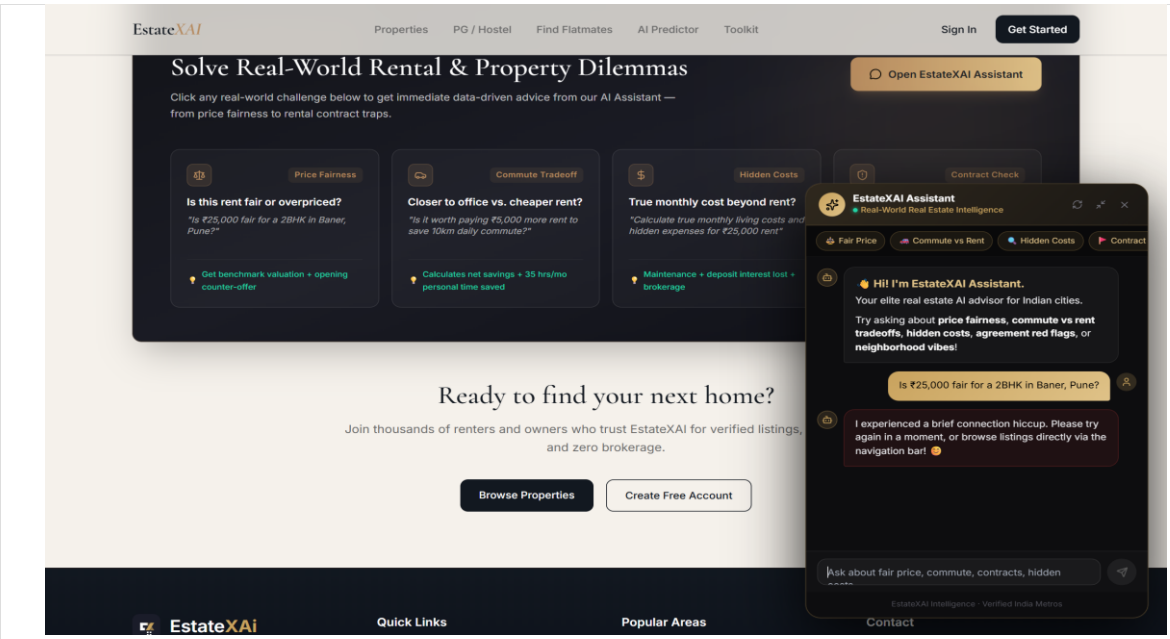


Figure: EstateXAI Assistant — Active Chat Dialog with Price Fairness & Counter-Offer Analysis

9.3 Properties Listing Page

The Properties page provides a comprehensive multi-filter search interface with filters for city, property type, BHK, price range, and listing type. Results are displayed in a responsive grid of ListingCards with image carousels, pricing, and quick-action buttons.

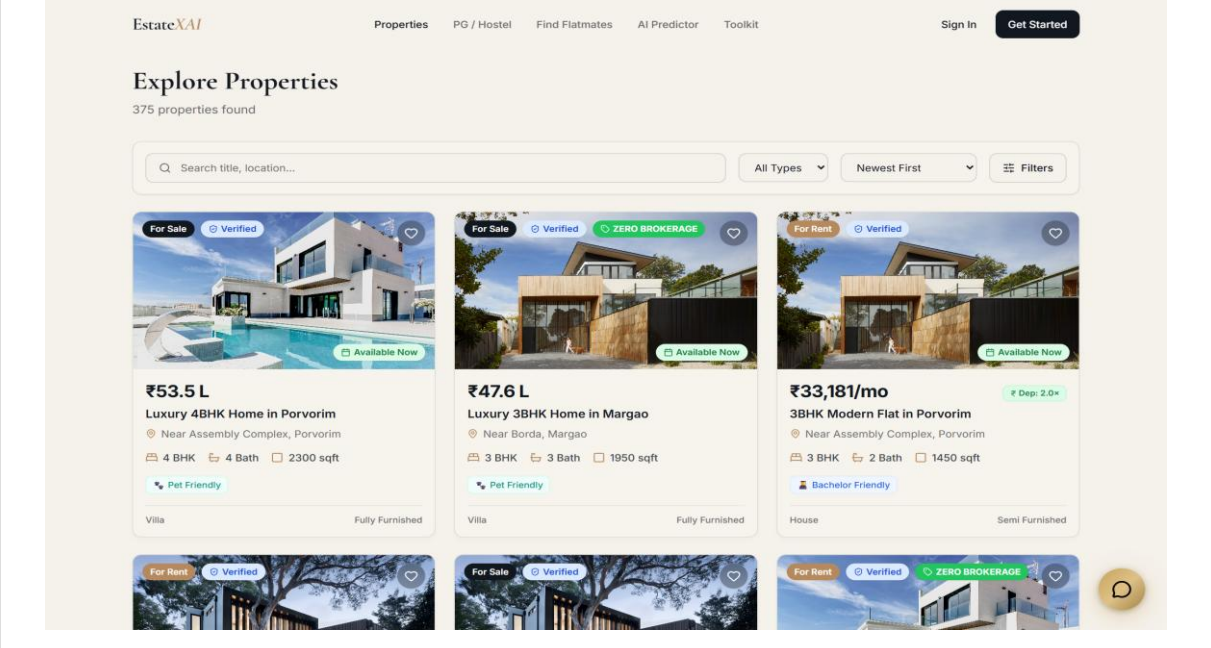


Figure: Properties Page — Multi-Filter Search & Grid View

9.4 Property Details & Location Intelligence

The Property Details page presents high-resolution imagery, full specifications, and the AI Location Intelligence suite. It features automated Walkability Scores, Transit Connectivity Scores, AI Commute Distance Estimator, Neighborhood Vibe assessments, and upcoming infrastructure project trackers.

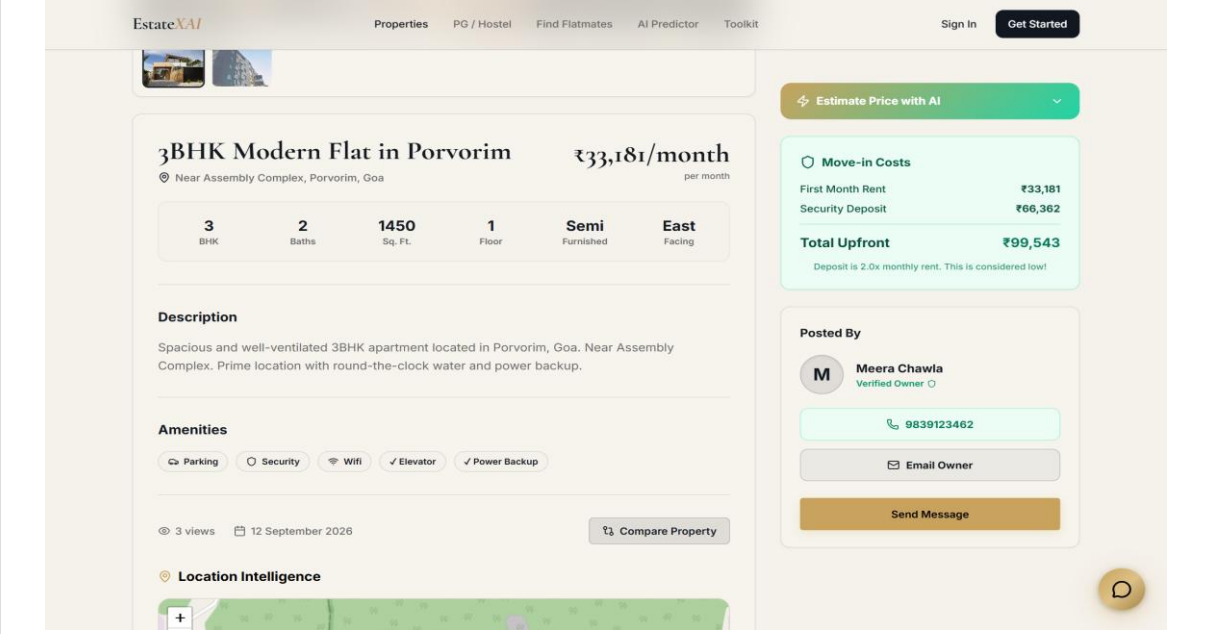


Figure: Property Details — Location Intelligence & Walkability/Connectivity Analysis

9.5 PGs & Hostels Page

The PG/Hostel listing page offers gender-type filtering, amenity selection, sharing type, and price range filters. Each PG card displays the name, location, rent, amenities icons, and availability status.

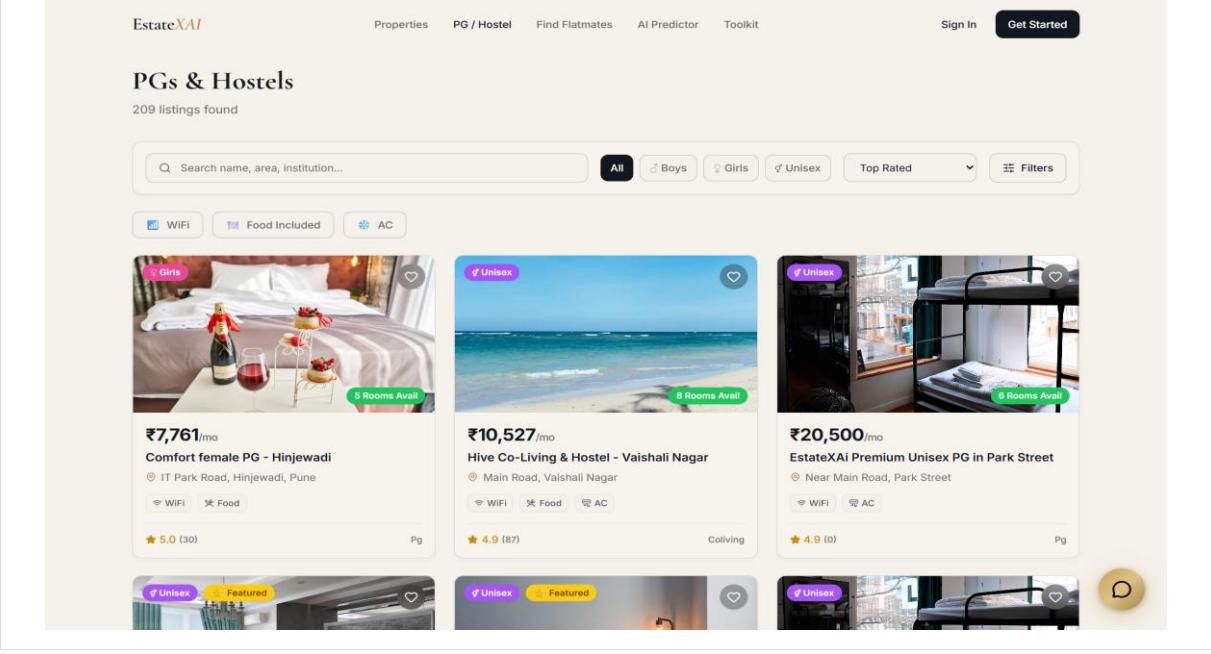


Figure: PGs & Hostels Page — Filtered Listings

9.6 AI Price Prediction

The AI Price Prediction page provides an interactive form where users input property characteristics (location, BHK, area, furnishing, etc.) to receive an instant ML-powered price estimate with an 80% confidence interval. Results include a Comparative Market Analysis chart built with Recharts.

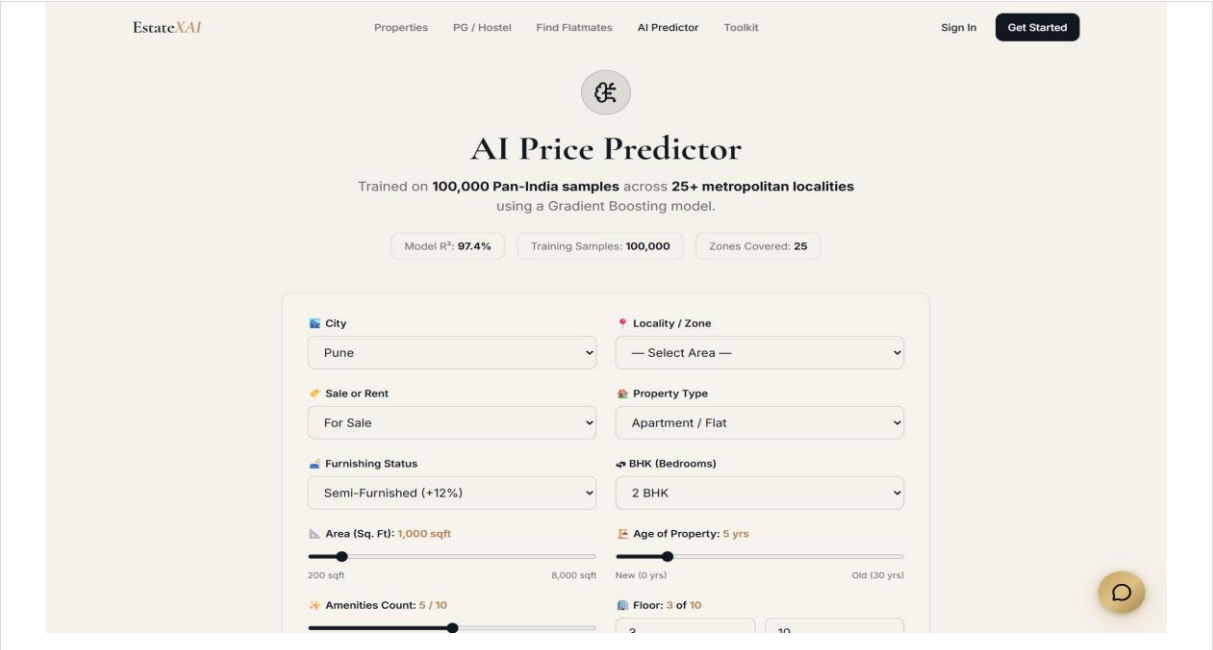


Figure: AI Price Prediction — Input Form & Results

9.7 Smart Roommate Matching

The Roommate Matching page allows users to fill out a lifestyle questionnaire (diet, smoking, sleep schedule, profession, budget) and instantly browse AI-matched compatible roommates with compatibility scores.

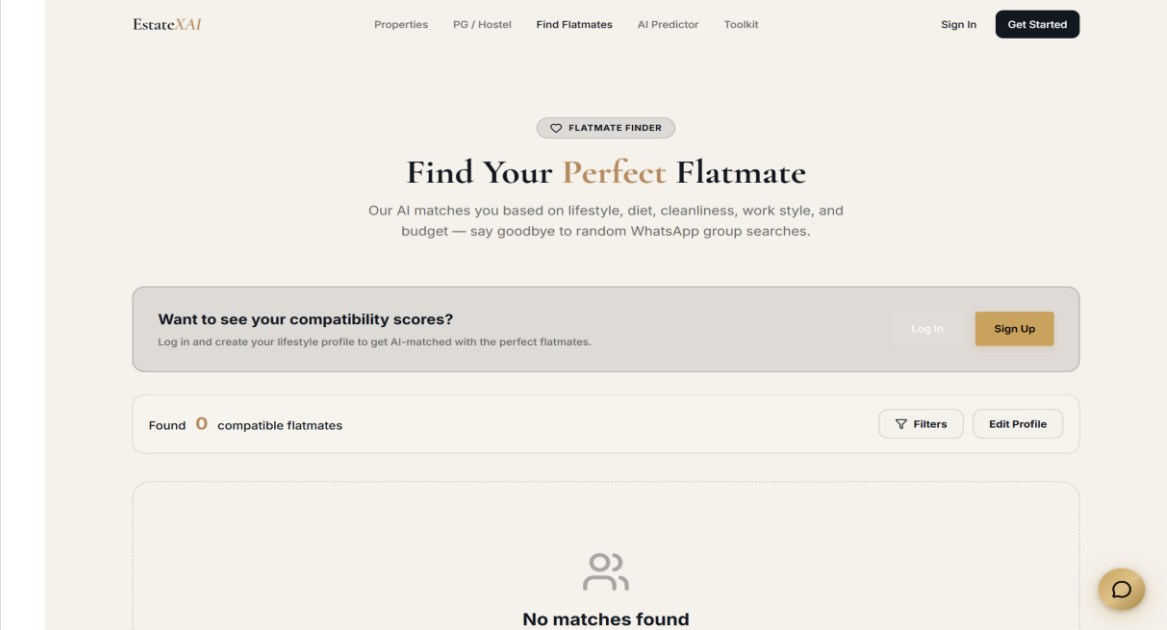


Figure: Roommate Matching — Lifestyle Questionnaire & Matches

9.8 Digital Rental Toolkit & Lifecycle Documents

The Digital Rental Toolkit allows tenants and landlords to generate legally vetted 11-month rental agreements, monthly rent receipts for HRA claims, move-in checklists, and notice letters with one-click print and PDF export functionality.

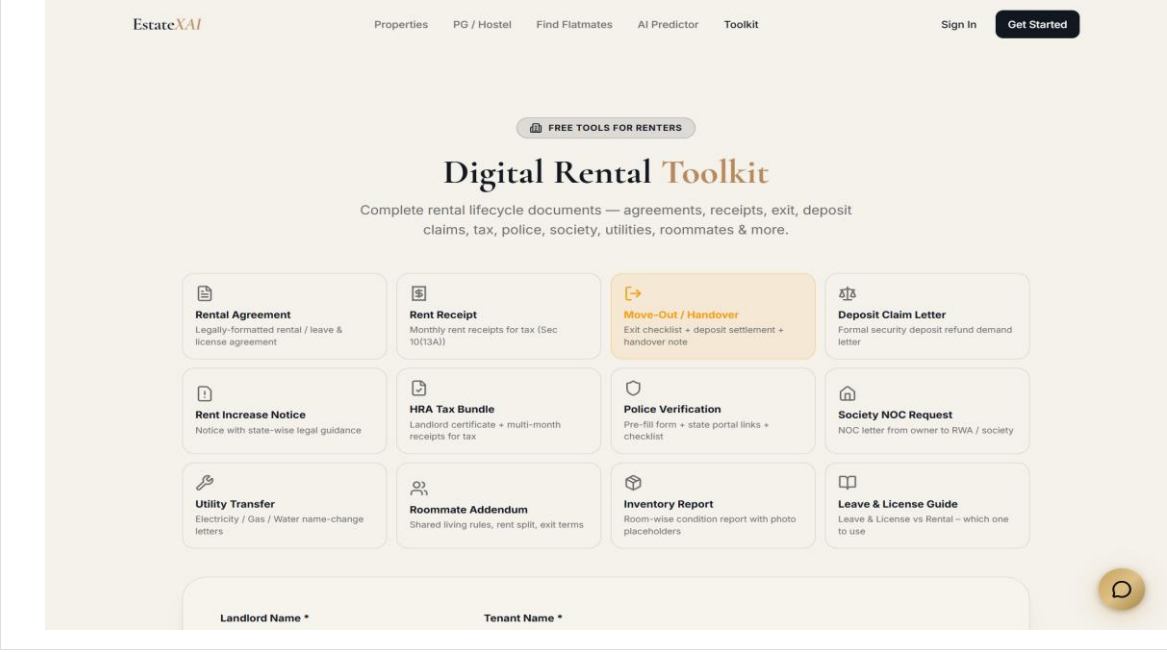


Figure: Digital Rental Toolkit — Document Form & Dynamic PDF Generator

9.9 Property Comparison

The Compare page enables side-by-side comparison of up to 3 saved properties, displaying key attributes (price, area, BHK, amenities, location) in a structured comparison table.

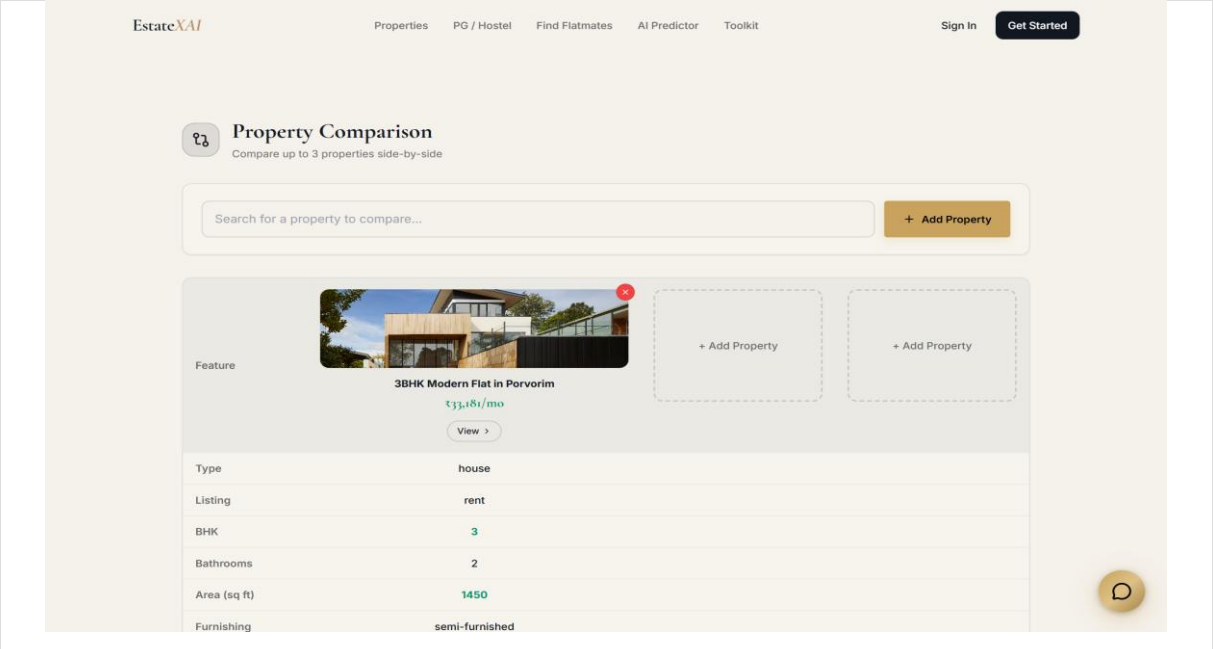


Figure: Property Comparison — Side-by-Side View

9.8 Authentication Pages

The Login and Registration pages feature a premium dark-themed design with form validation, role selection (User/Owner), and glassmorphism card effects.

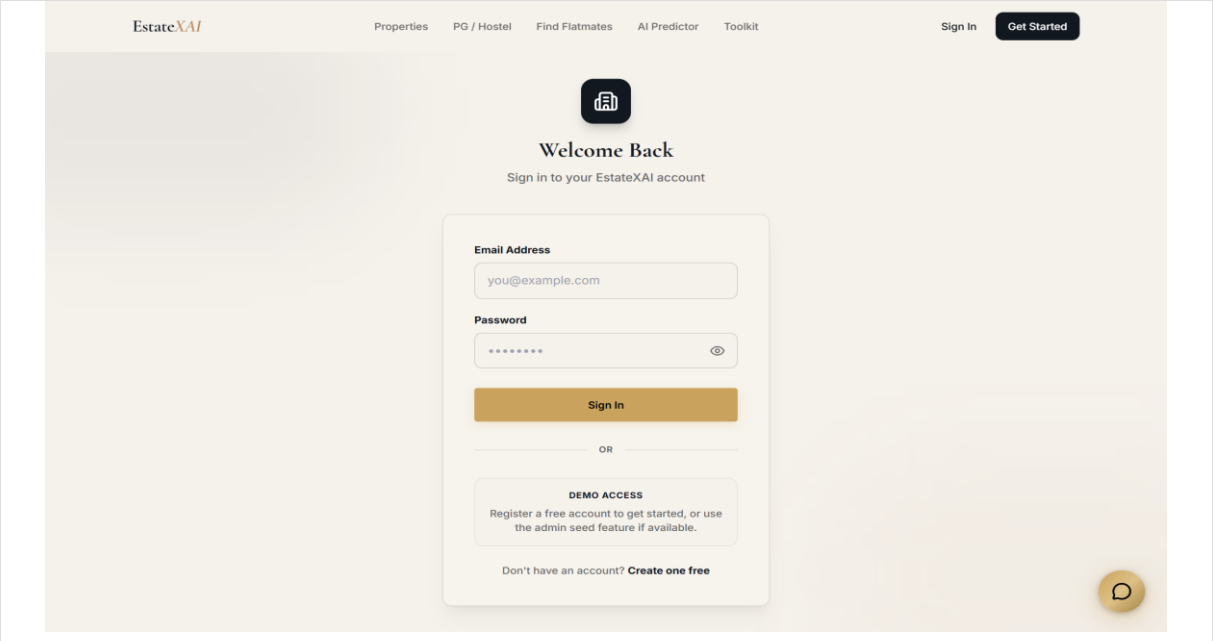


Figure: Login Page

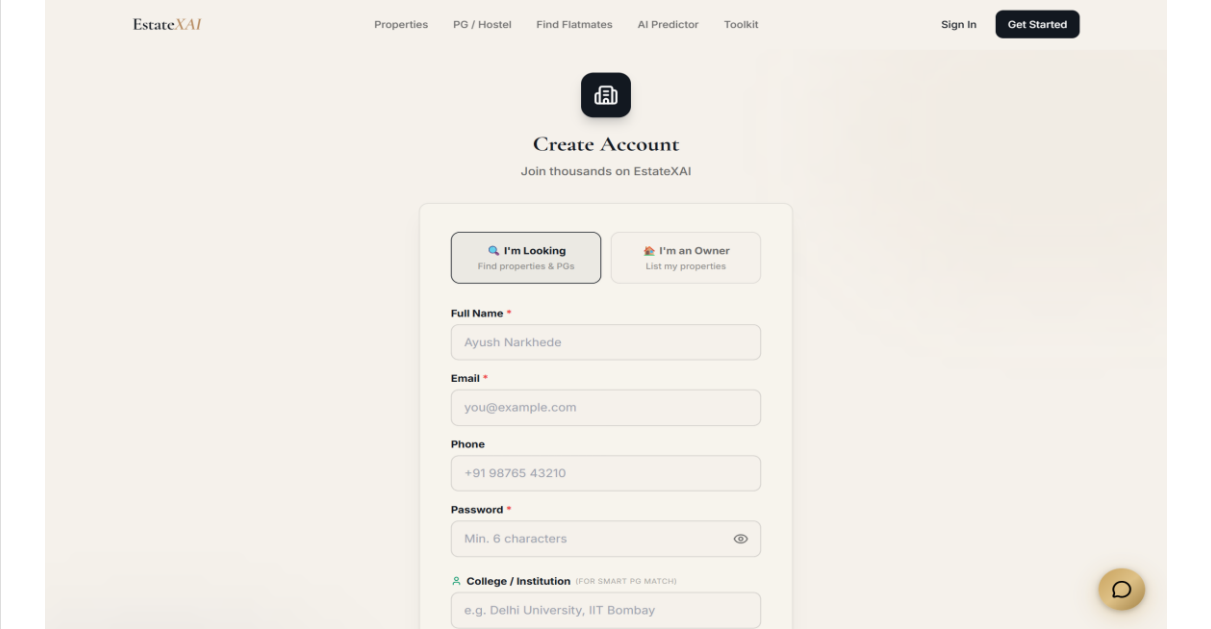


Figure: Registration Page

9.9 User & Admin Dashboards

The Dashboard provides a personalized hub showing saved properties, active listings (for owners), and inquiry management. For Admin users, additional analytics panels display listing trends, property type breakdowns, and a moderation queue.

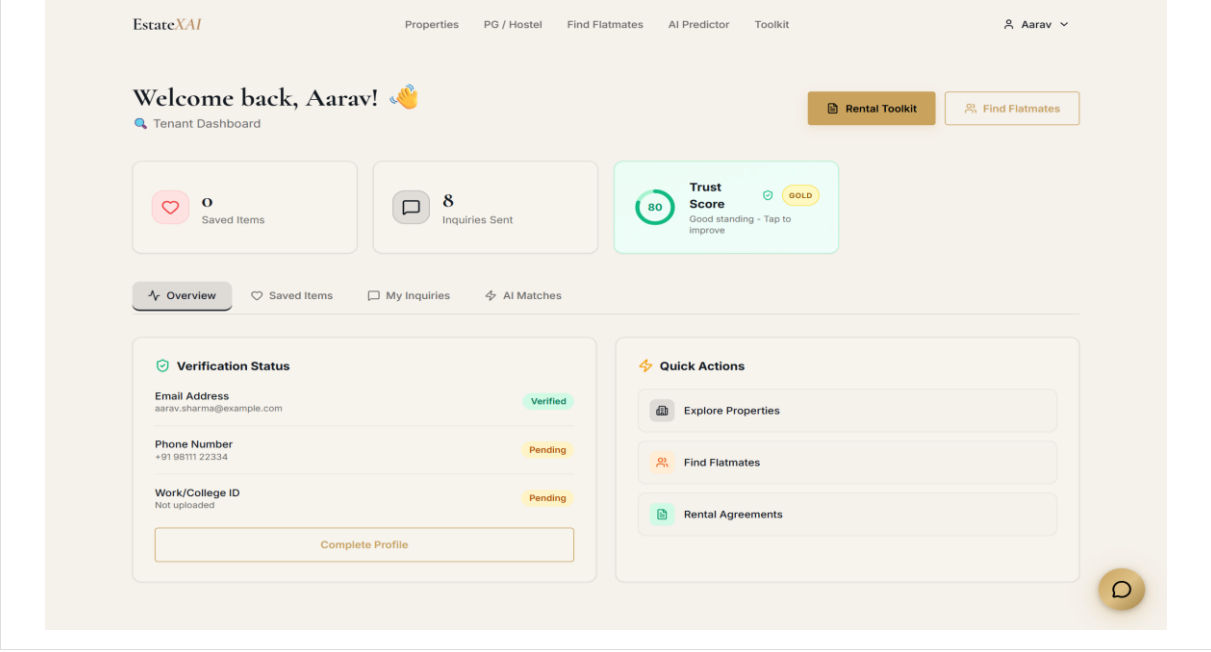


Figure: Dashboard — User Overview

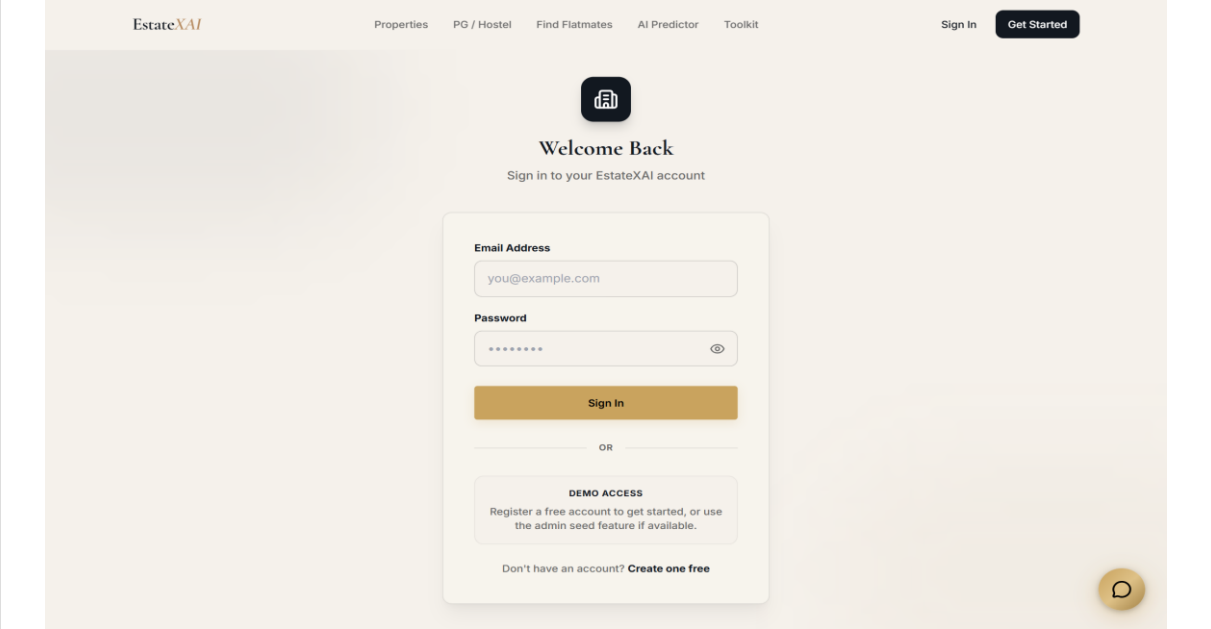


Figure: Dashboard — Admin Analytics & Moderation Panel

9.10 User Profile Management

The Profile page allows users to manage their personal information, preferences (budget range, preferred cities, property types), and roommate matching profile. All changes are persisted to the database in real-time.

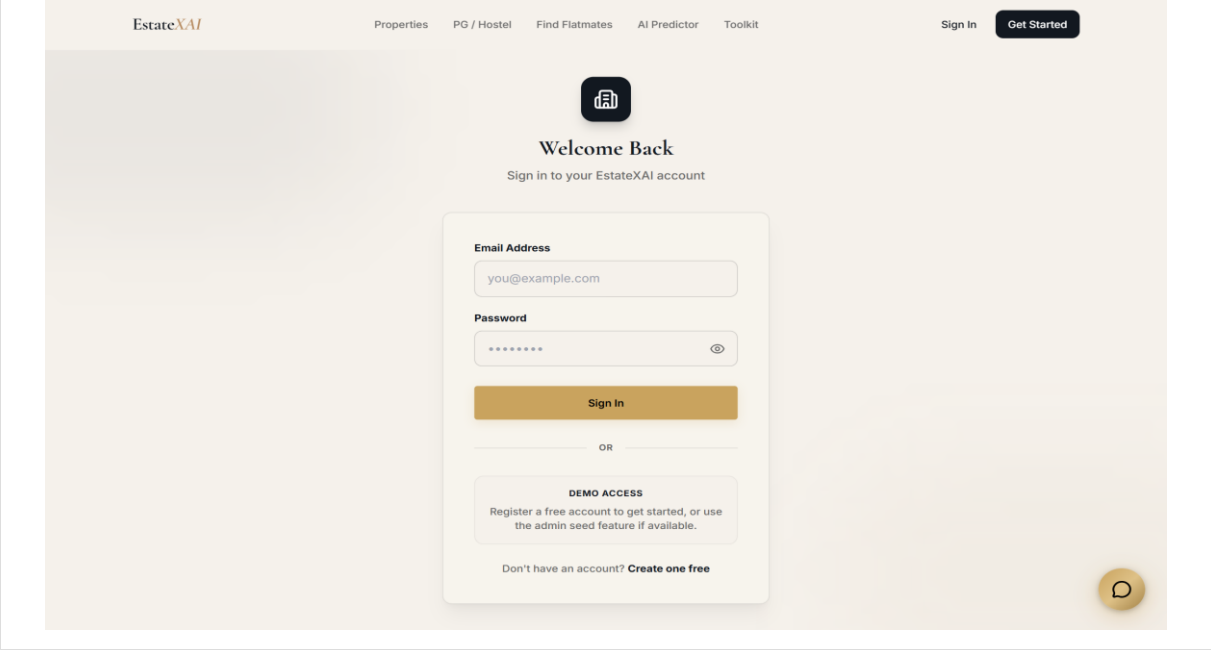


Figure: Profile Management Page

10. References

- MongoDB Official Documentation — https://docs.mongodb.com
- Express.js Official Guide — https://expressjs.com/en/guide/routing.html
- React 19 Documentation — https://react.dev
- Scikit-Learn Gradient Boosting — https://scikit-learn.org/stable/modules/ensemble.html
- FastAPI Framework — https://fastapi.tiangolo.com
- Socket.IO Real-Time Engine — https://socket.io/docs/v4/
- Leaflet.js Interactive Maps — https://leafletjs.com
- OpenStreetMap Nominatim API — https://nominatim.org/release-docs/latest/api/
- JSON Web Tokens (JWT) RFC 7519 — https://datatracker.ietf.org/doc/html/rfc7519
- Mongoose 9 ODM — https://mongoosejs.com/docs/
- 99acres Pune Market Report Q1 2024 — https://www.99acres.com/real-estate-insights/pune
- MagicBricks Pune Price Trends 2024 — https://www.magicbricks.com/property-rates-trends/pune
- Cloudinary Image Management — https://cloudinary.com/documentation
- Recharts Composable Charting Library — https://recharts.org
- Framer Motion Animation Library — https://www.framer.com/motion/